

Course Title: ADVANCED SOFTWARE ENGINEERING

Course Code: 24MCASET104

MODULE - II

Programming Style Guides and Coding Standards; Literate programming and Software documentation; Documentation generators, Javadoc Version control systems basic concepts; Concept of Distributed version control system and Git; Setting up Git; Core operations in Git version control system using command line interface (CLI): Clone a repository; View history; Modifying files; Branching; Push changes, Clone operation, add, commit, log. Pushing changes to the master; Using Git in IDEs and UI based tools. Software Quality: Understanding and ensuring requirements specification quality, design quality, quality in software development, conformance quality

Programming Style Guides and Coding Standards

In Advanced Software Engineering, Programming Style Guides and Coding Standards play a crucial role in writing maintainable, readable, and scalable code — especially in large teams and complex systems.

Programming Style Guide: A set of conventions about how code should look and be written.

Coding Standards: A broader set of rules and best practices that include both style and technical correctness, naming conventions, error handling, documentation, etc.

In advanced software engineering, programming style guides and coding standards serve as foundational tools to ensure uniformity, quality, and long-term maintainability across complex software systems. These documents define precise conventions for code structure, naming schemes, file organization, documentation, and language-specific constructs. In large-scale or enterprise-grade applications, where teams are distributed and codebases span millions of lines, such consistency becomes critical. It minimizes ambiguity, fosters shared understanding, and reduces the friction involved in code integration and maintenance. Style guides such as PEP 8 (Python), Google's C++ Style Guide, and Oracle’s Java Conventions are not just aesthetic preferences but are deeply rooted in principles of readability, clarity, and error prevention.

From a collaborative engineering standpoint, coding standards act as a contract among developers, establishing a common linguistic and structural framework. This promotes seamless collaboration by making individual contributions predictable and easy to comprehend, regardless of the author. Consistent coding practices also streamline code reviews and audits, facilitating better enforcement of software quality attributes such as maintainability, reliability, and reusability. Moreover, in advanced practices such as continuous integration and deployment (CI/CD), consistent code standards reduce merge conflicts and improve the efficiency of automated testing and static analysis tools, thereby accelerating development cycles without compromising quality.

Benefit	Explanation
Consistency	Everyone in the team writes code in a similar way.
Readability	Clean, consistent formatting makes it easier to read and understand.
Maintainability	Future developers can understand and update code without confusion.
Collaboration	Team members can work on the same codebase without conflict.
Avoids Bugs	Standards often include best practices that prevent common errors.
Automation	Easier to enforce checks via tools like linters, formatters, CI/CD pipelines.

Categories of Style Guides & Coding Standards

1. Naming Conventions

Helps make code self-documenting.

Element	Good Example	Bad Example
Variable	totalAmount	ta
Function	calculateTax()	calc()
Class	InvoiceGenerator	IG

Constant	MAX_RETRIES	mr
----------	-------------	----

2. Indentation and Spacing

Consistent indentation improves visual structure.

Good Approach

```
def calculate_total(items):  
    total = 0  
    for item in items:  
        total += item.price  
    return total
```

Bad Approach

```
def calculate_total(items):  
total=0  
    for item in items:  
        total+=item.price  
    return total
```

3. Commenting and Documentation

- Use comments to **explain "why"**, not "what".
- Use **docstrings** for functions and modules

Good Approach

```
def apply_discount(price, percentage):  
    """  
    Applies a percentage discount to the price.  
    """  
    return price - (price * percentage / 100)
```

Bad Approach

```
def apply_discount(price, percentage):  
    # subtract percentage from price  
    return price - (price * percentage / 100)
```

4. Code Structure and Modularity

- Write **small functions** that do one thing.
- Group related functions into classes or modules

Good Approach

```
public class UserService {  
    public void createUser(String name, String email) {  
        // create user  
    }  
  
    public void deleteUser(int userId) {  
        // delete user  
    }  
}
```

Bad Approach

```
public class Stuff {  
    public void doEverything() {  
        // too much going on  
    }  
}
```

5. Error Handling

Good Approach

```
try:  
    process_data()  
except ValueError as e:  
    print(f"Invalid input: {e}")
```

Bad Approach

```
process_data() # no error handling
```

Literate programming and Software documentation

In advanced software engineering, **literate programming** is a methodology introduced by Donald Knuth that emphasizes writing software as a form of communication between humans, rather than just instructions for machines. In this paradigm, code is interwoven with natural language explanations, allowing developers to document their thought process, design decisions, and algorithmic strategies in a coherent narrative. The result is a human-readable document that can be compiled into both executable code and rich documentation. This approach is particularly valuable in complex systems where understanding the rationale behind code is as important as the code itself, facilitating deeper insights into system behavior and design.

Software documentation, on the other hand, refers to the comprehensive set of materials that describe the functionality, design, and usage of software systems. In advanced engineering contexts, documentation is divided into several types: user documentation for end users, technical documentation for developers and maintainers, API documentation for integrators, and architectural documentation for system designers. Effective documentation ensures that software can be understood, used, modified, and extended long after its original authors are no longer involved. It also plays a key role in training, onboarding, system auditing, and regulatory compliance. When documentation is accurate and up-to-date, it reduces dependency on oral knowledge and tribal memory within teams.

Combining literate programming with rigorous documentation practices provides a powerful framework for building maintainable and transparent systems. Literate programming enriches technical documentation by integrating it directly with the codebase, eliminating the common problem of outdated or disconnected documentation. In environments such as scientific computing, high-assurance systems, or research-driven software, this approach enhances traceability, verification, and reproducibility. Advanced tooling (e.g., Jupyter Notebooks, Doxygen, Sphinx, and literate Haskell) supports literate programming and documentation generation, enabling teams to maintain clarity and consistency across rapidly evolving codebases. In summary, literate programming and software documentation are critical for fostering understanding, reducing errors, and supporting long-term sustainability in advanced software systems.

Literate Programming - Simple Example: Factorial (Python)

Instead of writing only code:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

In literate programming, we write something like this (Markdown + Python):

Calculating the Factorial of a Number

The factorial of a number n is defined as the product of all positive integers less than or equal to n . We define this recursively:

- If $n == 0$, then $\text{factorial}(n) = 1$ (base case)
- Otherwise, $\text{factorial}(n) = n * \text{factorial}(n-1)$

We can implement this as follows:

```
def factorial(n):

    if n == 0:

        return 1

    else:

        return n * factorial(n-1)
```

This approach helps developers understand **why** the code does what it does.

Tools used in literate programming:

- **Jupyter Notebooks** – great for Python (mix text + code)
- **noweb** or **Knuth’s WEB system**
- **RMarkdown** – for R, Python, etc.

Software Documentation (Advanced Software Engineering)

Documentation is essential in advanced software engineering practices for **maintenance**, **collaboration**, and **scalability**.

Type	Description	Example
User Manual	How to use the software	“How to log in to the system”
API Documentation	Describes classes, functions, endpoints	Swagger/OpenAPI, docstrings
System Documentation	Architecture, design decisions	UML diagrams, technical specs
Inline Comments	Clarify complex logic in code	# Sort the list by date
README	Overview, setup, usage	GitHub repos

A modern workflow might look like:

- Design your code using good engineering principles (e.g., SOLID, modularity)
- Write literate programs (mixing explanations + code)
- Add API documentation using docstrings
- Generate beautiful docs using tools (Sphinx, Swagger, etc.)
- Maintain README and system architecture docs

Concept	Key Point	Tool/Example
Literate Programming	Write code like a story	Jupyter, Org-mode
Software Documentation	Explain usage, design, and APIs	Sphinx, Swagger, Markdown
Simple Example	Factorial function, Calculator class	Explained above

Documentation generators, Javadoc

In advanced software engineering, **documentation generators** are tools that automatically create documentation from source code. These tools help developers maintain up-to-date and consistent documentation by extracting specially formatted comments and converting them into human-readable formats like HTML or PDF. This process saves time and reduces the risk of outdated manuals. A popular example of such a tool is **Javadoc**, used in Java programming. It parses Java source files and generates API documentation based on the structured comments written in the code.

Javadoc uses special tags within comment blocks, typically placed above classes, methods, or fields, to describe their purpose and behavior. For instance, a simple method might be documented like this:

```
/**  
  
 * Adds two numbers and returns the result.  
  
 * @param a the first number  
  
 * @param b the second number  
  
 * @return the sum of a and b  
  
 */  
  
public int add(int a, int b) {  
  
    return a + b;  
  
}
```

Javadoc will convert this comment into a formatted HTML page, making it easy for other developers to understand how to use the method without reading the source code directly.

In larger software projects, proper documentation is critical for maintainability, collaboration, and onboarding new developers. Using tools like Javadoc ensures that documentation evolves with the code and enforces a standard for clarity and consistency. Moreover, these tools can often be integrated into build systems (like Maven or Gradle), so documentation is generated automatically with every update. This makes Javadoc a valuable part of the software development lifecycle in Java-based applications.

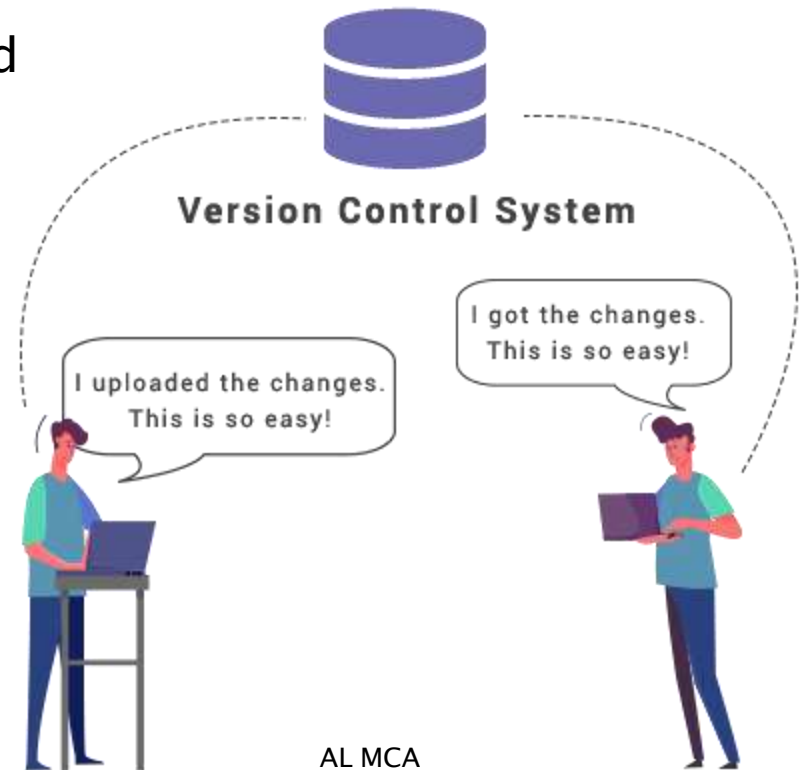
Version Control System



Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.

Version control is a type of system that allows you to keep track of changes made to your code over time. Version control is useful because:

- ❑ We can revert back to specific 'versions' of your code
- ❑ Collaboration on the same work is possible because specific changes and associated contributors are tracked



Types of VCS :

Centralized version control system (CVCS).

Distributed/Decentralized version control system (DVCS).

Centralized version control system (CVCS) uses a central server to store all files and enables team collaboration. But the major drawback of CVCS is its single point of failure

DVCS clients not only check out the latest snapshot of the directory but they also fully mirror the repository. If the server goes down, then the repository from any client can be copied back to the server to restore it

It is best practice to use version control to maintain source code and databases. It also streamlines collaborative projects. Changes can be recorded in a repository: a data structure that stores files and a record of changes made to those files.

Git is a distributed version-control system for tracking changes in source code during software development. It is designed for coordinating work among programmers, but it can be used to track changes in any set of files. Its goals include speed, data integrity, and support for distributed, non-linear workflows.

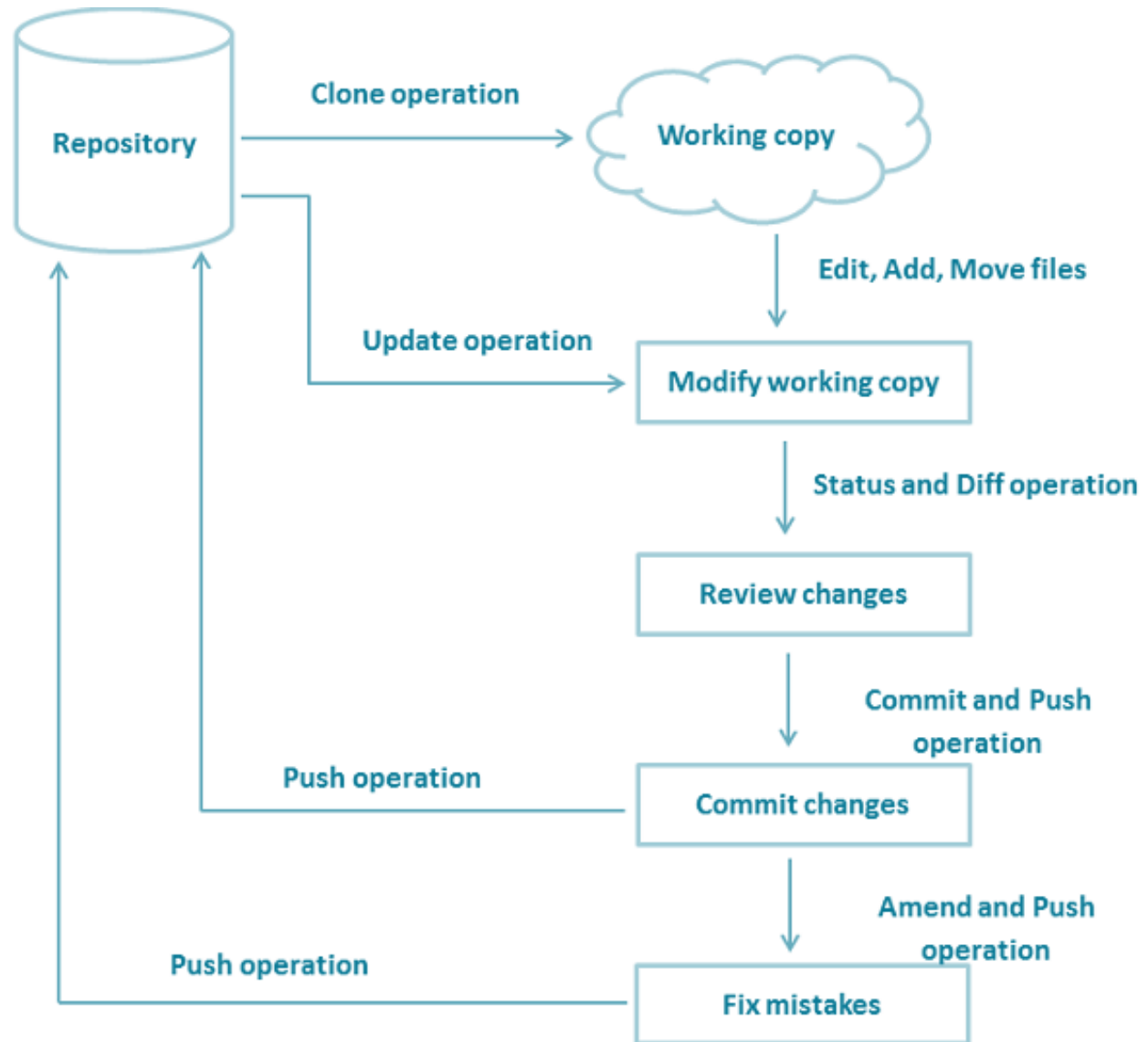
Git is one of the most popular version control systems. It is a distributed version control system. Changes do not have to be committed to the same central repository, which would require that every person working on the project to access that central repository and download the latest code in order to save changes. Instead, everyone can have their own localized repository with its entire history.

Repository

A repository, or Git project, encompasses the entire collection of files and folders associated with a project, along with each file's revision history. The file history appears as snapshots in time called commits, and the commits exist as a linked-list relationship, and can be organized into multiple lines of development called branches. Because Git is a DVCS, repositories are self-contained units and anyone who owns a copy of the repository can access the entire codebase and its history. Using the command line or other ease-of-use interfaces, a git repository also allows for: interaction with the history, cloning, creating branches, committing, merging, comparing changes across versions of code, and more.

Working in repositories keeps development projects organized and protected. Developers are encouraged to fix bugs, or create fresh features, without fear of derailing mainline development efforts. Git facilitates this through the use of topic branches: lightweight pointers to commits in history that can be easily created and deprecated when no longer needed. Through platforms like GitHub, Git also provides more opportunities for project transparency and collaboration. Public repositories help teams work together to build the best possible final product

General workflow



We can clone the Git repository as a working copy.



We can modify the working copy by adding/editing files.



If necessary, we can also update the working copy by taking other developer's changes.



We can review the changes before commit.



We can commit changes. If everything is fine, then we push the changes to the repository.



After committing, if we realize something is wrong, then we could correct the last commit and push the changes to the repository.

Characteristics of Git

Strong support for non-linear development



Distributed development



Compatibility with existent systems and protocols



Efficient handling of large projects



Cryptographic authentication of history



Toolkit-based design



Pluggable merge strategies

Advantages provided by Git is its flexibility in several aspects:

Track Changes – Changes can be tracked as someone making a change leaves a commit message about it.



Backup and Restore – It helps to maintain the source code backup.



Collaboration – It enables software team to collaborate with each other.



Branching and Merging – Changes are made on a branch and after being approved, they can be merged with the master branch. You can see who changed the file and what parts of the content are changed.



Deployment – It deploys the source code on the server with only one command.

The most popular Git solutions

Git suggests some repository management services that we can find useful. In general, repository management services are the key to fast and effective software development. The most popular ones based on Git are GitHub, Bitbucket and GitLab.

Git commands

These commands can be executed directly from the command line or by using an application like GitHub Desktop or Git Kraken. Here are some common commands for using Git:

- ❑ **git init** initializes a brand new Git repository and begins tracking an existing directory. It adds a hidden subfolder within the existing directory that houses the internal data structure required for version control.
- ❑ **git clone** creates a local copy of a project that already exists remotely. The clone includes all the project's files, history, and branches.

❑ **git add** stages a change. is a command, which adds changes in the working directory to the staging area. With the help of this command, you tell Git that you want to add updates to a certain file in the next commit. But in order to record changes, you need to run git commit too. Staging is a step before the commit process in git. That is, a commit in git is performed in two steps: staging and actual commit. As long as a changeset is in the staging area, git allows you to edit it as you like (replace staged files with other versions of staged files, remove changes from staging, etc.). Staging and committing separately gives developers complete control over the history of their project without changing how they code and work.

❑ **git commit** saves the snapshot to the project history and completes the change-tracking process. In short, a commit functions like taking a photo. Anything that's been staged with git add will become a part of the snapshot with git commit.

❑ **git status** shows the status of changes as untracked, modified, or staged.

❑ **git branch** shows the branches being worked on locally.

❑ **git merge** merges lines of development together. This command is typically used to combine changes made on two distinct branches. For example, a developer would merge when they want to combine changes from a feature branch into the master branch for deployment.

❑ **git pull** updates the local line of development with updates from its remote counterpart. Developers use this command if a teammate has made commits to a branch on a remote, and they would like to reflect those changes in their local environment.

❑ **git push** updates the remote repository with any commits made locally to a branch.

GitHub

❑ GitHub is a Git repository hosting service, but it adds many of its own features. While Git is a command line tool, GitHub provides a Web-based graphical interface. It also provides access control and several collaboration features

❑ Github is a web-based platform used for version control.

Features

Code review

Write better code

On GitHub, lightweight code review tools are built into every pull request. Your team can create review processes that improve the quality of your code and fit neatly into your workflow.

Start with a pull request

Pull requests are fundamental to how teams review and improve code on GitHub. Evolve projects, propose new features, and discuss implementation details before changing your source code.

Make a change

Start a new feature or propose a change to existing code with a pull request—a base for your team to coordinate details and refine your changes.

See every update

Discuss code

Resolve simple conflicts

You can't always avoid conflict. Merge pull requests faster by resolving simple merge conflicts on GitHub—no command line necessary.

Merge the highest quality code

Reviews can improve your code, but mistakes happen. Limit human error and ensure only high quality code gets merged with detailed permissions and status checks.

Bulletproof your review process

Build on GitHub with review tools to avoid human error and add extra polish to your team's code with review tools

Project management

Project management, made simple

On GitHub, project managers and developers coordinate, track, and update their work in one place, so projects stay transparent and on schedule.

Start with an issue

Create an issue to suggest a new idea or track a bug. Then organize and assign tasks to your team. And don't spare any attachments—issues support most image and file types.

Link issues and pull requests

Track updates to your project without complicating your workflow by crosslinking between issues and pull requests anywhere in your GitHub organization

See your work take shape with Projects

Visualize all of your work and prioritize it right alongside your code with projects boards. See what tasks are planned or in-progress, either in a repository or across your organization.

Personalized project management

Extend your project management process using tools that integrate with and build on GitHub issues and pull requests.

Integrations

Built to grow

Between applications and custom tools, there's no limit to the ways you can build on GitHub. Fine tune your process as your team changes, update tools as technologies shift, and find new ways to work better.

Discover new tools

From continuous integration to code review, find tools for every step of the development process and start using them in minutes.

Build on GitHub

Create your own tools with greater access to data than ever before using the same API that we use to build GitHub.

Actions

Automate your workflow from idea to production

Run a workflow on any GitHub event

Kick off workflows with GitHub events like push, issue creation, or a new release. Combine and configure actions for the services you use, built and maintained by the community.

Packages

Your packages, at home with their code

With GitHub Packages you can safely publish and consume packages within your organization or with the entire world.

Publish privately for your team, or publicly for the open source community

Discover and publish public and private packages in one place. Then seamlessly use and reuse any package as a dependency in a project by downloading it straight from GitHub.

Automate it all with GitHub Actions

Security

Securing software, together

Find vulnerabilities that other tools miss

GitHub Security Advisories

Security alerts

GitHub reviews every security vulnerability to identify and alert affected repositories.

Update vulnerable dependencies, automatically

Team management

The human side of software

Building software is as much about managing teams and communities as it is about code. Whether you're on a team of two or two thousand, we've got the support your people need.

Hosting

All your code and documentation in one place

There are hundreds of millions of private, public, and open source repositories hosted on GitHub. Every repository is equipped with tools to help you host, version, and release code and documentation.

The GitHub flow

The GitHub flow is a lightweight, branch-based workflow built around core Git commands used by teams around the globe—including ours.

The GitHub flow has six steps, each with distinct benefits when implemented:

- ❑ **Create a branch:** Topic branches created from the canonical deployment branch (usually master) allow teams to contribute to many parallel efforts. Short-lived topic branches, in particular, keep teams focused and results in quick ships.
- ❑ **Add commits:** Snapshots of development efforts within a branch create safe, revertible points in the project's history.
- ❑ **Open a pull request:** Pull requests publicize a project's ongoing efforts and set the tone for a transparent development process.

❑ **Discuss and review code:** Teams participate in code reviews by commenting, testing, and reviewing open pull requests. Code review is at the core of an open and participatory culture.

❑ **Merge:** Upon clicking merge, GitHub automatically performs the equivalent of a local 'git merge' operation. GitHub also keeps the entire branch development history on the merged pull request.

❑ **Deploy:** Teams can choose the best release cycles or incorporate continuous integration tools and operate with the assurance that code on the deployment branch has gone through a robust workflow.

Core operations in Git version control system using command line interface

Clone a repository

git clone : is used to get a copy of an existing Git repository

Git receives a full copy of nearly all data that the server has. Every version of every file for the history of the project is pulled down by default when you run git clone.

In fact, if your server disk gets corrupted, you can often use nearly any of the clones on any client to set the server back to the state it was in when it was cloned

You clone a repository with **git clone <url>**

if we want to clone the Git linkable library called libgit2, we can do so like this:

```
$ git clone https://github.com/libgit2/libgit2
```

That creates a directory named libgit2, initializes a .git directory inside it, pulls down all the data for that repository, and checks out a working copy of the latest version. If we go into the new libgit2 directory that was just created, we'll see the project files in there, ready to be worked on or used.

View history

git log <filename> : if you want to find history information about a README.md file in the local directory, run the following command:

git log README.md

git log lists the commits made in that repository in reverse chronological order; that is, the most recent commits show up first.

Modifying files

➤The **git add** command adds a change in the working directory to the staging area. It tells Git that you want to include updates to a particular file in the next commit. However, git add doesn't really affect the repository in any significant way—changes are not actually recorded until you run git commit

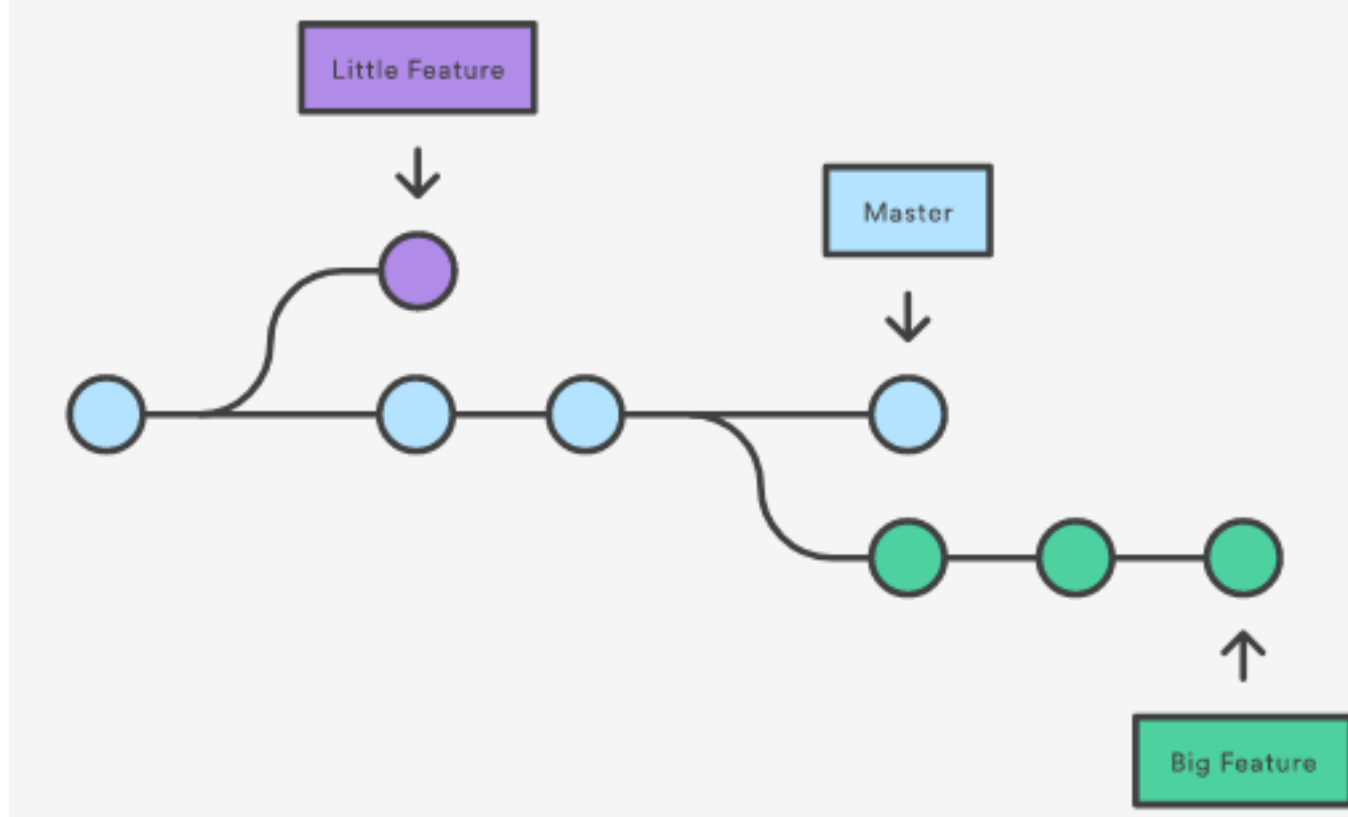
➤The **git status** command displays the state of the working directory and the staging area. It lets you see which changes have been staged, which haven't, and which files aren't being tracked by Git. Status output does not show you any information regarding the committed project history.

➤The **git rm** command can be used to remove individual files or a collection of files. The primary function of `git rm` is to remove tracked files from the Git index. Additionally, `git rm` can be used to remove files from both the staging index and the working directory.

Branching

A branch represents an independent line of development. Branches serve as an abstraction for the edit/stage/commit process.

The git branch command does more than just create and delete branches. If you run it with no arguments, you get a simple listing of your current branches



➤ The diagram in the previous slide visualizes a repository with two isolated lines of development, one for a little feature, and one for a longer-running feature. By developing them in branches, it's not only possible to work on both of them in parallel, but it also keeps the main master branch free from questionable code.

➤ The implementation behind Git branches is much more lightweight than other version control system models.

➤ We create a branch using the following command:

git branch sampleone

git branch

List all of the branches in your repository. This is synonymous with `git branch --list`.

git branch <branch>

Create a new branch called `<branch>`. This does not check out the new branch.

git branch -d <branch>

Delete the specified branch. This is a “safe” operation in that Git prevents you from deleting the branch if it has unmerged changes.

git branch -D <branch>

Force delete the specified branch, even if it has unmerged changes. This is the command to use if you want to permanently throw away all of the commits associated with a particular line of development.

git branch -m <branch>

Rename the current branch to `<branch>`.

git branch -a

List all remote branches.

Push changes

git push is most commonly used to publish an upload local changes to a central repository. After a local repository has been modified a push is executed to share the modifications with remote team members.

git push <remote> <branch>

Push the specified branch to , along with all of the necessary commits and internal objects. This creates a local branch in the destination repository.

git push <remote> --force

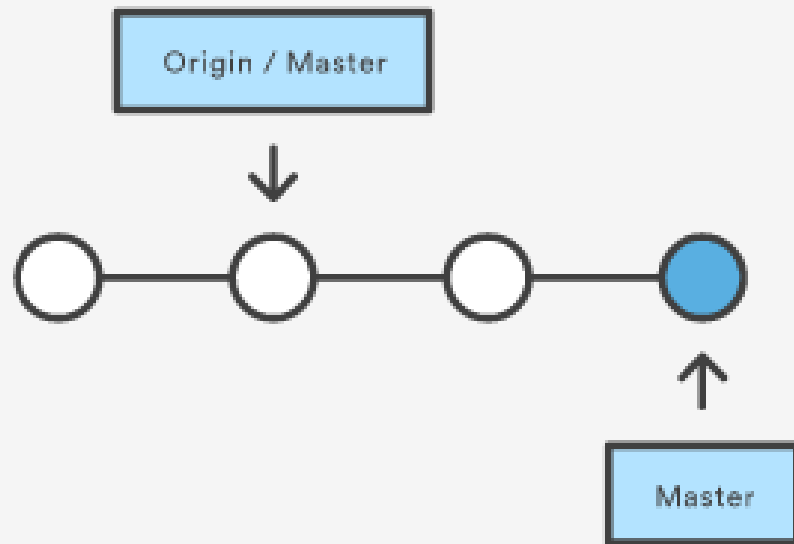
Same as the above command, but force the push even if it results in a non-fast-forward merge. Do not use the --force flag unless you're absolutely sure you know what you're doing.

git push <remote> --all

Push all of your local branches to the specified remote.

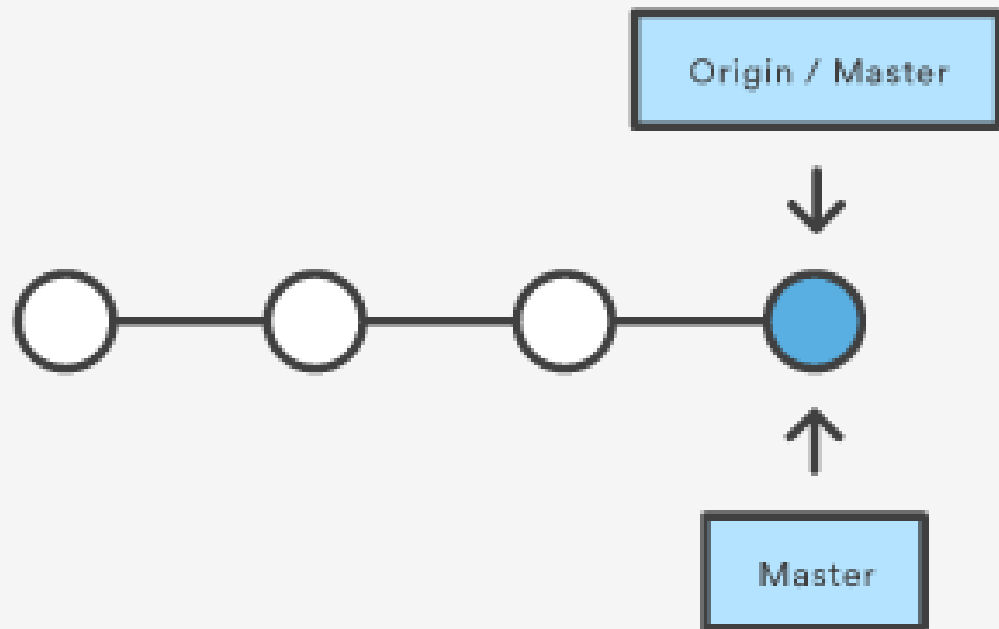
git push <remote> --tags

Tags are not automatically pushed when you push a branch or use the --all option. The --tags flag sends all of your local tags to the remote repository.



Before Pushing

After Pushing



The diagram in the previous slide shows what happens when your local master has progressed past the central repository's master and you publish changes by running `git push origin master`. Notice how `git push` is essentially the same as running `git merge master` from inside the remote repository.

add

➤ git add is the first command in a chain of operations that directs Git to "save" a snapshot of the current project state, into the commit history.

➤ The git add command adds a change in the working directory to the staging area. It tells Git that you want to include updates to a particular file in the next commit.

➤ git add works on the more abstract level of changes. This means that git add needs to be called every time you alter a file, whereas svn add only needs to be called once for each file. It may sound redundant, but this workflow makes it much easier to keep a project organized

E.g. **git add hello.py git commit**

git add

Stage all changes in <file> for the next commit.

git add

Stage all changes in <directory> for the next commit.

git add -p

Begin an interactive staging session that lets you choose portions of a file to add to the next commit. This will present you with a chunk of changes and prompt you for a command. Use **y** to stage the chunk, **n** to ignore the chunk, **s** to split it into smaller chunks, **e** to manually edit the chunk, and **q** to exit.

commit

- The `git commit` command captures a snapshot of the project's currently staged changes. Committed snapshots can be thought of as “safe” versions of a project—Git will never change them unless you explicitly ask it to.
- Commits are created with the `git commit` command to capture the state of a project at that point in time.
- In Git, repositories are distributed, Snapshots are committed to the local repository, and this requires absolutely no interaction with other Git repositories. Git commits can later be pushed to arbitrary remote repositories.

git commit

Commit the staged snapshot. This will launch a text editor prompting you for a commit message. After you've entered a message, save the file and close the editor to create the actual commit.

git commit -a

Commit a snapshot of all changes in the working directory. This only includes modifications to tracked files

git commit -m "commit message"

A shortcut command that immediately creates a commit with a passed commit message. Passing the -m option will forgo the text editor prompt in-favor of an inline message.

git commit -am "commit message"

A power user shortcut command that combines the -a and -m options. This combination immediately creates a commit of all the staged changes and takes an inline commit message.

git commit --amend

This option adds another level of functionality to the commit command. Passing this option will modify the last commit. Instead of creating a new commit, staged changes will be added to the previous commit. This command will open up the system's configured text editor and prompt to change the previously specified commit message.

e.g.

Let's make further updates to hello.py and execute the following:

git add hello.py

git commit --amend

This will once again, open up the configured text editor. This time, however, it will be pre-filled with the commit message we previously entered.

diff

Git diff is a command-line utility. It's a multiuse Git command. When it is executed, it runs a diff function on Git data sources. These data sources can be files, branches, commits, and more. It is used to show changes between commits, commit, and working tree, etc.

Different scenarios where we can utilize the git diff command.

- The usual use of git diff command that we can track the changes that have not been staged.
- The git diff command allows us to track the changes that are staged but not committed. We can track the changes in the staging area.

➤Track the changes after committing a file. Suppose we have committed a file for the repository and made some additional changes after the commit. So we can track the file on this stage also.

➤Git allows us to track changes between two commits, whether it is the latest commit or the old commit. But the required thing for this is that we must have a list of commits so that we can compare.

The git diff command allows us to compare different versions of branches and repository. To get the difference between branches, run the git diff command as follows:

```
$ git diff <branch 1> <branch 2>
```

The above command will display the differences between branch 1 and branch 2. So that you can decide whether you want to merge the branch or not

Conflict resolution

Git merge conflicts

- Sometimes multiple developers may try to edit the same content. If Developer A tries to edit code that Developer B is editing a conflict may occur.
- The git merge command's primary responsibility is to combine separate branches and resolve any conflicting edits.

Types of merge conflicts

A merge can enter a conflicted state at two separate points. When starting and during a merge process.

➤Git fails to start the merge

A merge will fail to start when Git sees there are changes in either the working directory or staging area of the current project. Git fails to start the merge because these pending changes could be written over by the commits that are being merged in.

A merge failure on start will output the following error message:

error: Entry '<fileName>' not uptodate. Cannot merge. (Changes in working directory)

➤Git fails during the merge

A failure DURING a merge indicates a conflict between the current local branch and the branch being merged. This indicates a conflict with another developers code. Git will do its best to merge the files but will leave things for you to resolve manually in the conflicted files. A mid-merge failure will output the following error message:

error: Entry '<fileName>' would be overwritten by merge. Cannot merge. (Changes in staging area)

How to identify merge conflicts ?

Git will produce some descriptive output letting us know that a CONFLICT has occurred. We can gain further insight by running the git status command

How to resolve merge conflicts using the command line ?

- The most direct way to resolve a merge conflict is to edit the conflicted file.
- Simply remove all the conflict dividers.
- Once the file has been edited use git add to stage the new merged content. To finalize the merge create a new commit.
- e.g.

```
git commit -m "merged and resolved the conflict in merge.txt"
```
- Git will see that the conflict has been resolved and creates a new merge commit to finalize the merge.

Git commands that can help resolve merge conflicts

General tools

git status

The status command is in frequent use when a working with Git and during a merge it will help identify conflicted files.

git log --merge

Passing the --merge argument to the git log command will produce a log with a list of commits that conflict between the merging branches.

git diff

diff helps find differences between states of a repository/files. This is useful in predicting and preventing merge conflicts.

Tools for when git fails to start a merge

git checkout

checkout can be used for undoing changes to files, or for changing branches

git reset --mixed

reset can be used to undo changes to the working directory and staging area.

Tools for when git conflicts arise during a merge

git merge --abort

Executing git merge with the --abort option will exit from the merge process and return the branch to the state before the merge began.

git reset

Git reset can be used during a merge conflict to reset conflicted files to a known good state

Pushing changes to the master

Git push origin master is a special command-line utility that specifies the remote branch and directory. When you have multiple branches and directory, then this command assists you in determining your main branch and repository.

Generally, the term origin stands for the remote repository, and master is considered as the main branch. So, the entire statement "git push origin master" pushed the local content on the master branch of the remote location.

Syntax:

```
$ git push origin master
```

Clone operation

In Git, cloning is the act of making a copy of any target repository. The target repository can be remote or local. You can clone your repository from the remote repository to create a local copy on your system.

Git Clone Command

The git clone is a command-line utility which is used to make a local copy of a remote repository. It accesses the repository through a remote URL.

Usually, the original repository is located on a remote server, often from a Git service like GitHub, Bitbucket, or GitLab. The remote repository URL is referred to the origin.

Syntax:

```
$ git clone <repository URL>
```

Software Quality: Understanding and ensuring requirements specification quality, design quality, quality in software development, conformance quality.

Software Quality shows how good and reliable a product is.

Software quality refers to the degree to which a software system meets functional requirements, non-functional requirements, user expectations, and industry standards.

It is commonly viewed from four aspects:

- Requirements Specification Quality
- Design Quality
- Quality in Software Development (Process Quality)
- Conformance Quality (Product Quality)



Portability: Software is claimed to be transportable, if it may be simply created to figure in several package environments, in several machines, with alternative code products, etc.

Usability: A software has smart usability if completely different classes of users (i.e. knowledgeable and novice users) will simply invoke the functions of the products.

Reusability: A software has smart reusability if completely different modules of the products will simply be reused to develop new products.

Correctness: Software is correct if completely different needs as laid out in the Software Requirement Specification (SRS) document are properly enforced.

Maintainability: A software is reparable, if errors may be simply corrected as and once they show up, new functions may be simply added to the products, and therefore the functionalities of the products may be simply changed, etc

Reliability: Software is more reliable if it has fewer failures. Since software engineers do not deliberately plan for their software to fail, reliability depends on the number and type of mistakes they make. Designers

can improve reliability by ensuring the software is easy to implement and change, by testing it thoroughly, and also by ensuring that if failures occur, the system can handle them or can recover easily.

Efficiency: The more efficient software is, the less it uses of CPU-time, memory, disk space, network bandwidth, and other resources. This is important to customers in order to reduce their costs of running the software, although with today's powerful computers, CPU time, memory and disk usage are less of a concern than in years gone by.

Evolution of Quality Management System

Quality Systems are basically evolved over the past some years. The evolution of a Quality Management System is a four-step process.

- **Inspection:** Product inspection task provided an instrument for quality control (QC).
- **Quality Control:** The main task of quality control is to detect defective devices, and it also helps in finding the cause that leads to the defect. It also helps in the correction of bugs.
- **Quality Assurance:** Quality Assurance helps an organization in making good quality products. It also helps in improving the quality of the product by passing the products through security checks.
- **Total Quality Management (TQM):** Total Quality Management(TQM) checks and assures that all the procedures must be continuously improved regularly through process measurements.

Requirements Specification Quality

A Software Requirements Specification (SRS) is considered high quality when it is:

Characteristics of a High-Quality SRS

1. Correct

The SRS must accurately represent the intended functionality.

Example:

If users need a "dark mode option", it must be explicitly mentioned. If missing, the SRS is incorrect.

2. Unambiguous

Every requirement must have only one interpretation.

Bad:

“System should load fast.” → (What is fast?)

Good:

“System should load the dashboard within 2 seconds under normal load.”

3. Complete

All necessary requirements must be included.

Example:

If a login feature is mentioned, the SRS must also describe:

- password rules
- validation

- error messages
- forgot password flow

4. Consistent

No contradictory requirements.

Contradiction Example:

- Req 12: User must log in to access reports.
- Req 19: Reports must be accessible without login.

5. Verifiable

Requirements should be testable.

- Non-verifiable: "The UI should be beautiful."
- Verifiable: "UI must follow Material Design Guidelines."

6. Feasible

The requirement must be technically and economically achievable.

7. Traceable

Each requirement must be traceable through design, code, and testing artifacts.

Example:

Requirement R1 → Design Module D1 → Code files C1,C2 → Test cases T1,T2

8. Modifiable

Easy to update with minimal ripple effects.

Example of a Good Requirement:

RQ-01: "The system shall allow users to reset their password via email link within 5 minutes of request."

Design Quality

Design transforms requirements into a structured solution. High design quality means the design is:

1. Functional (Supports Requirements)

Every requirement should have a corresponding design element.

2. Cohesion (High Cohesion is Good)

Each module should perform one well-defined task.

Example:

A "PaymentProcessor" class should only process payments, not manage user profiles.

3. Coupling (Low Coupling is Good)

Modules should be minimally dependent on each other.

Example:

A UI component should not directly access database tables; it should call a middle-layer API.

4. Abstraction

Hiding unnecessary details.

Example:

Using a “DatabaseInterface” rather than exposing SQL queries directly.

5. Modularity

Breaking the system into independent, replaceable components.

Example:

Shopping cart, product catalog, and user account modules function independently.

6. Scalability

The design should handle increased user load.

Example:

Using a microservices architecture for a large e-commerce system.

7. Reusability

Components should be easily reused in other projects.

Example:

Authentication module reusable across multiple applications.

8. Security Compliance

Design must consider:

- access control
- encryption
- error handling

Example of Good Design

Instead of one large “UserManagement” class, break into:

- UserRegistration
- UserAuthentication
- UserProfileManager

This improves cohesion, modularity, and testability.

Quality in Software Development (Process Quality)

This focuses on how the software is developed (process, methods, tools).

1. Use of Standards

Following ISO 9001, CMMI, IEEE standards.

2. Code Quality

Good development practices include:

Readability

Proper naming conventions: `calculateTotalPrice()`

Maintainability

Well-structured code with comments and clear logic.

Robustness

Handles invalid inputs gracefully.

Example:

If user uploads an unsupported file format, system must show a friendly error.

3. Version Control Practices

Using Git branches, pull requests, code reviews.

4. Testing Practices

- Unit testing
- Integration testing
- System testing
- Regression testing
- Automated testing
- Continuous integration (CI)

5. Documentation

API documentation, coding guidelines, architecture documents.

6. Developer Skill and Training

Proper training and adherence to best practices improves output quality.

7. Continuous Integration / Continuous Deployment (CI/CD)

Automated build and test pipelines ensure early detection of issues.

Example of Good Process Quality

A team uses:

- GitHub actions for CI
- Automated unit tests with 85% coverage
- Coding guidelines
- Weekly code reviews

This ensures fewer bugs and higher maintainability.

Conformance Quality (Product Quality)

Conformance quality means the final product meets requirements and design specifications.

It is measured during testing and after deployment.

Measures of Conformance Quality

1. Reliability

Software should run without failure for a long period.

Example:

ATM software running 24/7 without frequent crashes.

2. Performance

Meeting speed, latency, and throughput requirements.

Example:

Video streaming app should support 1080p streaming without lag.

3. Usability

Easy for users to operate.

4. Security

System should protect against:

- unauthorized access
- SQL injection
- data breaches

5. Maintainability

Easy to update or modify after deployment.

6. Portability

Runs on multiple platforms with little change.

Example:

Java applications that run on Windows, macOS, Linux.

7. Compliance

Meets:

legal regulations (e.g., GDPR- General Data Protection Regulation)

industry standards

Example of Conformance Quality Failure

Requirement: “App must load within 2 seconds.”

Actual product: Loads in 7 seconds.

→ Poor conformance quality, even if code is clean.